GigBid: R&D report

AI-4016 R&D assignment, design review. Repo `github.com/awwab-03/gigbid`, live `https://40-233-123-21.sslip.io`. Every number cites a file in `evals/results/` or a commit.

1. Problem and constraints

A freelancer spends hours a week reading job posts, discarding scams and poor fits, pricing, and writing proposals that prove relevant experience. GigBid automates the reading, judging and drafting and keeps the human on every action that has consequences.

Never automated: submitting a proposal, spending Connects, sending a client message, accepting a contract. Each is an `interrupt()` in a LangGraph graph (`agent/graphs/bid_graph.py:79`, `conversation_graph.py:49`), the autonomy setting has no "auto-send" level (`agent/config.yaml: autonomy`), and live mode never scrapes or uses browser automation (`mcp_servers/marketplace/adapters/upwork_live.py`).

Declared budgets (`agent/config.yaml: budgets`): p95 latency 25 s per job analysed and 90 s per proposal; cost USD 0.004 per job and 0.02 per proposal; at most 40 graph steps, 2 Critic revisions, 60k tokens per job. Open models on Cerebras' free tier cost USD 0, so tokens are reported alongside.

Data. No simulator or labelled data was supplied, so we built GigSim and a synthetic, seeded (seed 7) world: 3 personas with portfolios, 840 jobs, 600 with gold labels (bid/skip and reasons, scam type, planted injections, gold evidence, fair price band). `docs/DATA_CARD.md` documents generation and its limits; the most important limit is that GigSim's simulated client is a rule-based model we wrote, so "reply rate" measures agreement with those rules, not with real clients.

2. Architecture

Bid graph (one thread per job; rendered from code in `docs/images/bid_graph.mmd`): `intake` fans out with `Send` to **Job Analyst**, **Risk Assessor** and **Fit Matcher** in parallel; they join at `decide` (deterministic rules), then `pricing` (deterministic quote + Connects expected value), `writer`, `verifier` (claim verification), `critic` (at most 2 revisions), `compliance`, `approval` (`interrupt`) and `submit` (idempotent). **Conversation graph** (one thread per client thread): classify intent, find calendar slots, draft reply, compliance, approval, send.

Agent	Why it exists separately	Model
Job Analyst	turns an untrusted post into a typed `JobSpec` the rest can rely on	small (heuristic fallback)
Risk Assessor	scam and injection checks must run before anything is drafted; rules first, model only when ambiguous	small, rarely
Fit Matcher	requirement-to-evidence matrix is retrieval, not generation	none (Portfolio MCP)
decide / pricing	deterministic and testable (ADR 0005)	none
Writer	grounded drafting with `[E#]` citations	large
Verifier	removes unsupported claims before the Critic sees them	cross-encoder
Critic	independent quality review, bounded loop	large
Compliance Guard	platform rules, contact details, guarantees, policy version	small + rules

Marketplace port. Agents never call GigSim directly: the Marketplace MCP server has a `MarketplaceBackend` port with a GigSim adapter and an Upwork-MCP adapter (`mcp_servers/marketplace/adapters/`). **MCP surface:** 22 tools on three servers (`docs/TOOLS.md`), per-agent allowlists (`agent/config.yaml: tool_allowlist`), bearer service tokens, write tools that require idempotency keys and support `dry_run`, a hash-chained audit log per server, schema snapshot tests. **Stores:** Postgres for LangGraph checkpoints, the LangGraph store (style and outcome memory) and app tables; pgvector for portfolio vectors. **Deployment** (ADR 0008): one Oracle Always Free VM (1 GB), MCP servers in-process, inline worker and scheduler, Caddy HTTPS.

3. R&D question A: does tool granularity change agent behaviour?

Hypothesis. A few coarse tools with an `action` enum lower tool-selection errors (fewer choices) but raise argument errors (one schema for many intents); rewriting our descriptions with "use for / not for" guidance helps most on the ambiguous pairs (search vs match vs verify, client-history facets).

Setup. 40 tasks with a known correct first call (`evals/runners/tool_granularity.py`), four surfaces: *fine* (~36 narrow tools), *coarse* (3 tools with `action`), *ours* (the real 22 MCP tools, schemas read from the servers) and *ours_rw* (same tools, rewritten descriptions). Two models. Scored on the first call: tool-selection accuracy, argument accuracy on key arguments, tokens per task (API usage).

TBD: table from `evals/results/tool_granularity.json`, conclusion.

4. R&D question B: RAG variants, claim verification, what ships

Hypothesis (assignment form). Requirement decomposition plus hybrid retrieval raises evidence Recall@5 by at least 15 points over V0 on jobs with four or more requirements.

Retrieval (450 dev jobs, `evals/results/rag_variants.json`):

Variant	R@5 all	nDCG@10 all	R@5, 4+ reqs (n=368)	R@5 vague posts (n=15)	R@5 injected posts (n=48)
V0 naive dense	0.637	0.713	0.641	0.562	0.566
V2 hybrid	0.748	0.787	0.749	0.783	0.739
V3 hybrid + rerank	0.786	0.816	0.784	0.787	0.746
V8 corrective RAG	**0.798**	**0.828**	**0.796**	0.738	**0.777**

V1 0.716, V4 0.750 and V5 0.646 R@5 are in the same file. **Result:** on 4+ requirement jobs V8 beats V0 by 15.5 points, V2 by 10.8, so the hypothesis holds for V8, not for decomposition + hybrid alone. Two negative results: multi-vector (V5) barely beats V0, and on vague posts V8's Recall@5 is *below* V2/V3 (its nDCG is best: it ranks the one right item first and retrieves fewer of the others). By persona V8 R@5 is 0.785 / 0.820 / 0.790 (ML / full-stack / data).

Claim verification. Claims are split from the draft and graded against retrieved evidence with the cross-encoder; the supported/unsupported threshold was chosen by Youden's J on labelled pairs (-7.9 logit, AUC 0.787, `support_threshold.json`). TBD: unsupported claims per proposal before/after from the proposal evals.

Downstream. Decisions on the same 150 jobs and model: TBD (post-fix V8 vs V0, `decision_eval_open_*_fixed`). Proposals, judge, reply rate, latency, tokens: TBD (`proposal_eval_qwen_v8_open_qwen`, `proposal_eval_qwen_v0_open_qwen`). Judge calibration: TBD (`judge_calibration.json`, 30 items labelled by the author).

What ships. V8 (ADR 0004): best retrieval on every overall metric and on 4+ requirement and injected posts, p95 4.1 s vs V3's 7.5 s. The 1 GB live VM runs V2 because V8's cross-encoder overloaded it (measured: 20 cross-encoder pairs 1.5 s, load average 11); claim verification stays on there.

5. R&D question C: single agent vs prebuilt supervisor vs our graph

TBD: setup recap and table from `evals/results/agent_comparison.json` (51 dev jobs, same read-only tools, same simulated client scoring, qwen-3.8-27b for all three because gpt-oss-120b's free tier allows 5 requests/minute).

6. The change request

TBD: needs the 2 November change-request text. The design absorbs policy changes through `GET /policy` (Connects multiplier, fees, submission caps, length rules are read at run time by Pricing and Compliance).

7. Signature extension: X2 proposal A/B test

Arm A evidence-led (shipped writer) vs arm B plan-first (`agent/prompts/writer_plan.md`), same jobs, model, retrieval and simulator; hypothesis registered in `docs/EXTENSION.md` before arm B ran (commit d7d6f94). TBD: results from `evals/results/ab_test_x2.json`.

8. Failure analysis

The V8 decision eval (`decision_eval_open_v8_limit50_rows.jsonl`, 150 jobs) had **19 wrong decisions, all bids on gold-skip jobs**, and 32 of 112 correct bids priced outside the fair band (28 below).

Category	Jobs (examples)	Cause	Fix tried	Worked?
Wrong bid: budget below floor	5 (j-00003 $24/h vs $30 floor; j-00044 $160 vs $300 min)	`profile` missing from `BidState`; LangGraph dropped it, so the floor was 0	declare `profile` in the state; regression test (commit f9badfa)	TBD
Mispriced (below band)	28 (j-00008, j-00015, j-00023: $28/h, band from $39)	same bug: pricing used its default $20 floor x 1.4	same fix	TBD
Wrong bid: over-generous evidence	9 (j-00005, j-00027, j-00206, j-00227: gold coverage 0.4, predicted 5/7 requirements "supported")	retrieval finds loosely related evidence for missing skills (Next.js, GraphQL, Unity)	none yet; candidate: stricter support threshold on fit	no
Missed evidence	26 jobs wrong with V0 and right with V8 (8 the other way)	dense retrieval misses paraphrased requirements	V8 corrective RAG	yes (net +18)
TBD	unsupported claims, compliance misses, injection, tool errors	from proposal evals and injection eval		

9. Safety and platform respect

GigBid must never submit, send or spend without approval; never contact clients off-platform; never fabricate experience; never follow instructions inside job posts or messages; never scrape. Enforcement: interrupts on every write, idempotency keys, per-agent tool allowlists, claim verification + Critic + Compliance Guard, untrusted-text delimiting and quarantine (`agent/guardrails.py`), rate limits far below human use in live mode (RUNBOOK). **What still gets through:** TBD from `injection_eval_open_qwen.json` (the regex detector catches all 71 planted dev-set injections with 0 false positives on 529 clean posts, but 0 of 12 paraphrased attacks).

10. Results against declared budgets

TBD. The hidden test set runs after the deadline; this section reports the dev-set measurements against the declared budgets, and the hidden-test numbers when they are released.